

What we're going to cover:

- Explaining difference between `sort()` and `sorted()`
- Error handling
- Catching specific exceptions
- Raising custom errors

`sort()`

این متد تنها بر روی لیست‌ها قابل اعمال است

`sorted()`

این روش را می‌توان برای هر نوع شیء قابل تکرار (مانند تاپل‌ها، دیکشنری‌ها و غیره) به کار برد

یکی دیگر از تفاوت‌های عمده این است که

- `sort()`
- متد لیست موجود را در حالی که تغییر می‌دهد، اصلاح می‌کند
- `sorted()`
- این متد یک فهرست مرتب‌شده برمی‌گرداند

```
In [2]: arr = [3, 4, 6, 1, 0, 7, 8]
arr.sort()
print(arr)
print('--' * 10)
arr_2 = [3, 4, 6, 1, 0, 7, 8]
k = sorted(arr_2)
print(arr_2)
print(k)
```

```
[0, 1, 3, 4, 6, 7, 8]
```

```
-----
```

```
[3, 4, 6, 1, 0, 7, 8]
```

```
[0, 1, 3, 4, 6, 7, 8]
```

```
In [3]: data = [('Alice', 30), ('Bob', 25), ('Charlie', 35)]
sorted_data = sorted(data, key=lambda tup: tup[1]) # tup[1] sort based on numbers
print(sorted_data)
print('--' * 10)
fruit_dict = {'apple':20, 'orange':12, 'banana':36, 'watermelon':1, 'grapes':100}
sorted_dict = dict(sorted(fruit_dict.items(), key=lambda x: x[1], reverse=True))
print(sorted_dict)
```

```
[('Bob', 25), ('Alice', 30), ('Charlie', 35)]
```

```
-----
```

```
{'grapes': 100, 'banana': 36, 'apple': 20, 'orange': 12, 'watermelon': 1}
```

```
In [2]: def division(a, b):
        return a/b

print(division(10, 4))
division(10, 0)
```

2.5

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[2], line 5
      1 def division(a, b):
      2     return a/b
      3
      4 print(division(10, 4))
----> 5 division(10, 0)

Cell In[2], line 2, in division(a, b)
      1 def division(a, b):
----> 2     return a/b

ZeroDivisionError: division by zero
```

```
In [3]: def division(a, b):
        try:
            return a/b
        except:
            print('Error in division')

print(division(10, 4))
division(10, 0)
```

2.5

Error in division

```
In [4]: def division(a, b):
        try:
            return a/b
        except Exception as e:
            print(f'Error is: {e}')

print(division(10, 4))
division(10, 0)
```

2.5

Error is: division by zero

```
In [5]: def division(a, b):
        try:
            return a/b
        except ZeroDivisionError:
            print('Zero division error.')
        except Exception as e:
            print(f'Error is: {e}')

print(division(10, 4))
```

```
division(10, 0)
division(12, 'abs')
```

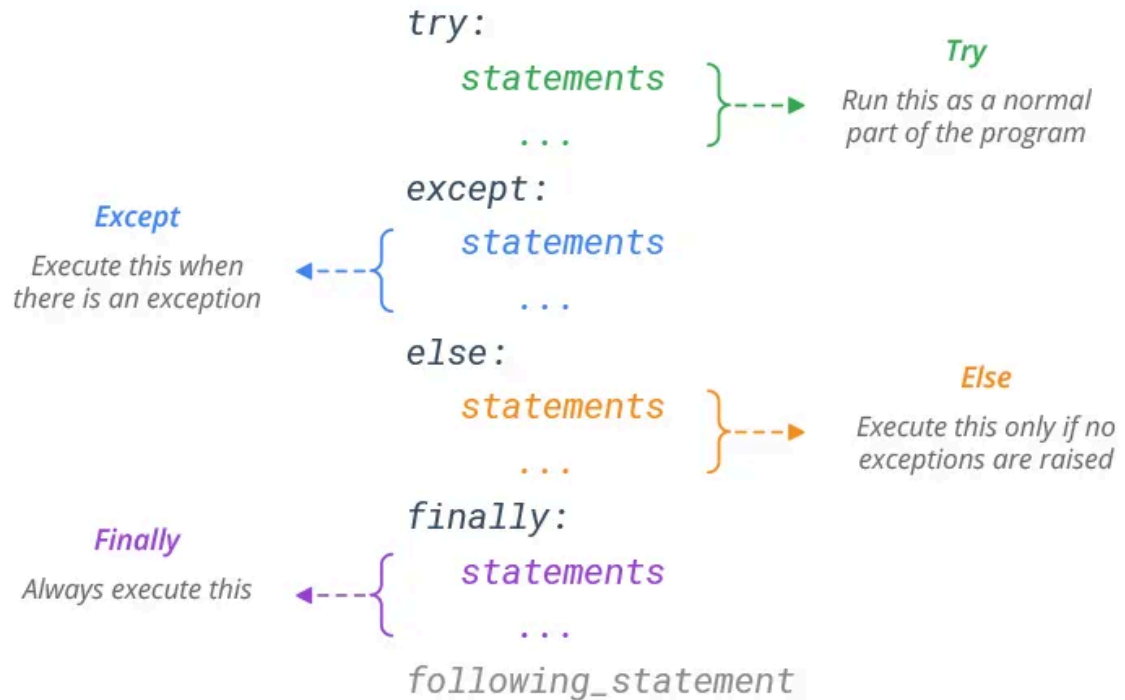
2.5

Zero division error.

Error is: unsupported operand type(s) for /: 'int' and 'str'

مدیریت خطاها و استثناءها

مدیریت مناسب خطاها یا استثناءها در پایتون، بخش مهمی از ساخت برنامه‌های مستحکم است



استثناهای رایج

در اینجا به برخی از استثناهای رایجی اشاره می‌کنیم که هنگام استفاده از توابع داخلی با آن‌ها مواجه خواهید شد

- **TypeError:**
- زمانی صادر می‌شود که عملیات یا تابعی بر روی شیئی با نوع نامناسب اعمال شود
- **ValueError:**
- زمانی صادر می‌شود که تابع آرگومانی با نوع صحیح، اما مقدار نامناسب دریافت کند
- **IndexError:**

- هنگامی که تلاش می‌شود به اندیسی خارج از محدوده در یک لیست دسترسی پیدا شود، این خطا رخ می‌دهد
- `KeyError`:
- زمانی که کلیدی در دیکشنری یافت نشود، صادر می‌شود
- `ZeroDivisionError`:
- هنگام تلاش برای تقسیم بر صفر، صادر می‌شود

Using try and except

`try`

برای بررسی خطاها در کد

`except`

برای مدیریت استثناها استفاده کنید

```
In [20]: def convert_to_integer(user_input):
    try:
        number = int(user_input)
        print(f'Result of function: {number}')
    except ValueError:
        # Handle non-integer input
        print("Error: Invalid input! Please enter a valid integer.")

    # Example usage
    convert_to_integer(50.22)
    convert_to_integer("abc") # This will trigger the ValueError
```

Result of function: 50

Error: Invalid input! Please enter a valid integer.

Using the Raise Statement

هنگام وقوع یک شرط خاص، عمداً استثنا ایجاد کنید

```
In [51]: def set_age(age):
    if age < 0:
        raise ValueError("Age can't be negative")
    print(f"Age is {age} years")

    print(set_age(-5))
```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[51], line 6
      2     if age < 0:
      3         raise ValueError("Age can't be negative")
      4     print(f"Age is {age} years")
      5
----> 6 print(set_age(-5))
      7 )

Cell In[51], line 3, in set_age(age)
      1 def set_age(age):
      2     if age < 0:
----> 3         raise ValueError("Age can't be negative")
      4     print(f"Age is {age} years")

ValueError: Age can't be negative

```

```

In [21]: def divide_numbers(a, b):
        if b == 0:
            # Raise an exception if b is zero
            raise ValueError("Cannot divide by zero!")
        return a / b

    try:
        # This will trigger the exception
        result = divide_numbers(10, 0)
    except ValueError as e:
        print(f"Error: {e}")

    # You can also test with a valid division
    try:
        result = divide_numbers(10, 2)
        print(f"The result is: {result}")
    except ValueError as e:
        print(f"Error: {e}")

```

Error: Cannot divide by zero!
The result is: 5.0

Using try, except, else, and finally

```

In [32]: try:
        n = 0
        res = 100 / n
    except ZeroDivisionError:
        print("You can't divide by zero!")
    except ValueError:
        print("Enter a valid number!")
    else:
        print("Result is", res)
    finally:
        print("Execution complete.")

```

You can't divide by zero!
Execution complete.